

İLERİ JAVASCRIPT VE ETKİLEŞİMLİ WEB



İÇİNDEKİLER

- Dom Manipülasyonu
- Olay Yönetimi (Event Handling)
- Basit Uygulama Örnekleri
- Sonuç Ve Değerlendirme



HEDEFLER

- Bu üniteyi çalıştıktan sonra;
 - DOM kavramını tanıyacak ve yapısını açıklayabilecek.
 - JavaScript ile DOM üzerinde öge seçme, ekleme ve silme işlemleri yapabilecek.
 - Farklı olay türlerini (click, hover, input vb.) ayırt edebilecek.
 - Event handling yöntemlerini kullanarak dinamik davranışlar ekleyebilecek.
 - Basit uygulamalar geliştirebilecek (liste, form doğrulama, içerik değiştirme).
 - Dinamik web sayfalarının kullanıcı deneyimine katkısını kavrayacak.
 - Saf JavaScript ile etkileşimli sayfalar geliştirme becerisi kazanacak.



Atatürk Üniversitesi
Açıköğretim Fakültesi

WEB TASARIMININ TEMELLERİ

Arş. Gör. Muhammed
Coşkun IRMAK

ÜNİTE 9

GİRİŞ

Günümüz web teknolojilerinde kullanıcı beklentileri yalnızca bilgiye erişimle sınırlı değildir; kullanıcılar aynı zamanda etkileşimli, dinamik ve kişiselleştirilmiş deneyimler talep etmektedir. Bu bağlamda JavaScript, web sayfalarına hareketlilik, etkileşim ve işlevsellik kazandıran en temel programlama dili olarak öne çıkmaktadır. Önceki üniteye ele alınan temel JavaScript kavramları (değişkenler, veri tipleri, operatörler ve basit etkileşimler) kullanıcıların programlama mantığını anlamalarına yardımcı olmuştur. Ancak modern web geliştirme süreçlerinde yalnızca bu temel kavramlar yeterli değildir. Gerçek hayattaki web uygulamalarında kullanıcı arayüzü, kullanıcıdan gelen girdilere anında tepki verebilmeli ve sayfa içerisindeki öğeleri dinamik olarak yönetebilmelidir.

Bu noktada Document Object Model (DOM) kavramı devreye girer. DOM, bir web sayfasındaki tüm öğeleri hiyerarşik bir ağaç yapısı içerisinde temsil eder ve geliştiricilerin JavaScript aracılığıyla bu öğelere erişmesine, üzerinde değişiklik yapmasına ve yeni öğeler eklemesine olanak tanır. Böylece kullanıcı ile web sayfası arasında dinamik bir etkileşim kurulabilir. Örneğin, bir butona tıklandığında sayfa içeriğinin değişmesi, kullanıcı bir form alanına veri girdiğinde anında doğrulama yapılması veya fareyle üzerine gelindiğinde bir görselin büyütülmesi gibi işlevler, DOM manipülasyonu ve olay yakalama (event handling) teknikleriyle gerçekleştirilmektedir.

JavaScript'in sunduğu event (olay) mekanizması, kullanıcı davranışlarını web sayfasına anlamlı bir şekilde aktarmayı sağlar. Olaylar, kullanıcı tarafından tetiklenen (tıklama, yazı yazma, kaydırma, fareyi üzerine getirme vb.) ya da tarayıcı tarafından gerçekleştirilen (sayfanın yüklenmesi, hata oluşması gibi) eylemler olabilir. Etkin bir web geliştirici, bu olayları yakalayarak uygun fonksiyonlarla cevap vermeyi öğrenmelidir. Böylece durağan bir sayfa yerine kullanıcıyla etkileşime giren, tepki veren ve dinamik olarak güncellenen bir arayüz tasarlanabilir.

Bu ünitenin amacı, öğrencilerin temel JavaScript bilgisini bir üst seviyeye taşıyarak etkileşimli web sayfaları geliştirme yetkinliği kazanmalarını sağlamaktır. Ünite kapsamında öncelikle DOM'un yapısı ve işleyişi ele alınacak, ardından çeşitli olay türleri ve bu olayların nasıl yakalanıp yönetileceği incelenecektir. Öğrenciler, örnek uygulamalar aracılığıyla butonlarla içerik değiştirme, dinamik liste oluşturma, form doğrulama ve görsel etkileşimler gibi temel interaktif işlemleri deneyimleyeceklerdir. Böylece yalnızca teorik bilgi edinmekle kalmayacak, aynı zamanda uygulamalı çalışmalar sayesinde modern web geliştirme sürecine doğrudan katkı sağlayacak beceriler kazanacaklardır.

Sonuç olarak, bu üniteyle birlikte öğrenciler, bir web sayfasının sadece tasarımsal yönünü değil, aynı zamanda kullanıcı etkileşimlerini de yönetebilecek düzeye geleceklerdir. Bu kazanımlar, sonraki ünitelerde ele alınacak olan çerçeveler (*frameworks*) ve kütüphaneler için sağlam bir temel oluşturacak; aynı

zamanda öğrencilerin kendi başlarına işlevsel, kullanıcı dostu ve etkileşimli web siteleri geliştirebilmelerine olanak tanıyacaktır.

DOM MANİPÜLASYONU

Web sayfalarının etkileşimli ve dinamik hale gelmesini sağlayan en önemli yapı Document Object Model (DOM)'dur. JavaScript'in sunduğu en güçlü özelliklerden biri, DOM üzerinden sayfa öğelerine erişim ve bu öğeler üzerinde değişiklik yapabilme yeteneğidir. DOM manipülasyonu sayesinde geliştirici, kullanıcı etkileşimlerine cevap verebilen, anlık olarak güncellenebilen ve daha işlevsel bir web deneyimi oluşturabilir. Bu bölümde DOM'un temel yapısı, öğelere erişim yöntemleri ve bu yapı üzerinde gerçekleştirilebilecek değişiklikler ayrıntılı olarak ele alınacaktır.

DOM'un Tanımı ve Yapısı

Web tasarımının dinamik boyutunu anlamak için öncelikle DOM'un temel mantığını kavramak gerekir. DOM, bir web sayfasının tarayıcı tarafından yorumlandıktan sonra bellekte oluşturulan hiyerarşik temsildir. Başka bir ifadeyle DOM, HTML veya XML belgelerini nesne tabanlı bir yapıya dönüştürerek geliştiricilerin bu yapıya programatik olarak erişmesini ve üzerinde değişiklik yapmasını sağlar.

Tarayıcı bir web sayfasını yüklediğinde, öncelikle HTML belgesini satır satır okur, yorumlar ve ardından bu belgeyi *düğüm*lerden (*nodes*) oluşan bir *ağaç yapısı* (*tree structure*) halinde belleğe aktarır. Bu yapı, sayfa içerisindeki her öğeyi (etiket, metin, öznitelik, yorum) birer düğüm (node) olarak temsil eder. Örneğin, <html> kök düğümü altında <head> ve <body> düğümleri bulunur; <body> içerisinde paragraf, başlık, bağlantı veya liste gibi öğeler yer alır. Böylece sayfa, mantıksal olarak dallanan bir yapı biçiminde düzenlenir.

DOM'un en önemli özelliği, dil bağımsızlığıdır. Yani DOM, yalnızca JavaScript ile sınırlı değildir; teorik olarak herhangi bir programlama dili DOM yapısını kullanabilir. Ancak modern web tarayıcılarında DOM ile en yaygın ve pratik etkileşim, JavaScript üzerinden gerçekleştirilir. Bu nedenle web geliştirme sürecinde "DOM manipülasyonu" dendiğinde genellikle JavaScript ile DOM'un kontrol edilmesi anlaşılır.

DOM üç temel bileşenden oluşur:

1. **Düğüm (Node):** Her HTML ögesi bir düğüm olarak temsil edilir. Örneğin <p> etiketi bir element düğümüdür. Metin içerikleri ise ayrı birer metin düğümüdür.
2. **Ağaç Yapısı (Tree Structure):** Düğümler hiyerarşik olarak bir ağaç yapısında düzenlenir. Bu hiyerarşi, ebeveyn (parent) ve çocuk (child) ilişkileri üzerinden tanımlanır.
3. **Arabirim (API):** DOM, geliştiricilerin düğümlere erişmesini ve değişiklik yapmasını sağlayan fonksiyonlar ve özellikler bütünüdür. Örneğin

`document.getElementById("id")` gibi yöntemler DOM API'nin parçalarıdır.

Şekil ile Açıklama (Metinsel Tasvir)

Bir HTML belgesinin DOM yapısını şu şekilde düşünebiliriz:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Örnek Sayfa</title>
  </head>
  <body>
    <h1>Merhaba Dünya</h1>
    <p>Bu bir paragraftır.</p>
  </body>
</html>
```

Bu kodun DOM karşılığı, aşağıdaki ağaç yapısı biçiminde temsil edilir:

- html
 - head
 - title → "Örnek Sayfa"
 - body
 - h1 → "Merhaba Dünya"
 - p → "Bu bir paragraftır."

Bu yapıdan da anlaşılacağı üzere, DOM her etiketi, her metni ve her ögeyi ayrı bir nesne olarak bellekte tutar. Geliştirici bu nesneler üzerinde oynama yaparak web sayfasını kullanıcı etkileşimlerine göre değiştirebilir.

DOM'un Web Geliştirme Açısından Önemi

DOM'un önemi birkaç temel noktada özetlenebilir:

- **Dinamik Güncelleme:** HTML statik bir dildir, ancak DOM sayesinde içerikler kullanıcı eylemlerine göre anlık olarak değiştirilebilir.
- **Kullanıcı Etkileşimi:** Kullanıcı bir butona tıkladığında ya da bir form alanına veri girdiğinde DOM aracılığıyla sayfa buna tepki verebilir.
- **Evrensel Standart:** DOM, W3C tarafından standartlaştırılmıştır, bu nedenle tüm modern tarayıcılarda aynı temel yapı geçerlidir.
- **Programatik Kontrol:** DOM, geliştiricilere bir belgenin yapısını, stilini ve içeriğini kod aracılığıyla kontrol etme olanağı sağlar.

Örnek: DOM'a Erişim

JavaScript ile DOM'a erişmek oldukça basittir. Örneğin aşağıdaki kod, sayfadaki `<h1>` etiketini seçip rengini kırmızıya dönüştürür:

```
let baslik = document.querySelector("h1");
baslik.style.color = "red";
```

Bu basit işlem, statik bir HTML belgesini anında dinamik bir görünüme dönüştürür. İşte bu nokta, modern web tasarımının temelini oluşturur.

DOM'da Ögelere Erişim Yöntemleri

DOM manipülasyonunun ilk ve en temel adımı, web sayfasındaki öğelere erişimdir. Tarayıcı tarafından oluşturulan DOM ağacındaki her düğüm, JavaScript aracılığıyla seçilebilir ve üzerinde değişiklik yapılabilir. Ancak hangi öğenin seçileceği ve nasıl erişileceği, geliştiricinin kullanacağı yöntemlere bağlıdır. Bu noktada DOM API, çeşitli fonksiyonlar ve seçici (selector) yöntemleri sunar. Her bir yöntem, farklı kullanım senaryolarına hitap eder ve doğru yöntem seçimi hem kodun verimliliğini hem de anlaşılabilirliğini artırır.

getElementById()

Tanım: Belirli bir id değerine sahip öğeyi seçmek için kullanılır.

Özellik: Tekil bir sonuç döndürür çünkü bir sayfada aynı id değeri yalnızca bir öğede bulunmalıdır.

Örnek:

```
<p id="paragraf1">Bu bir paragraftır.</p>
<script>
  let p = document.getElementById("paragraf1");
  p.style.color = "blue";
</script>
```

Bu kod, *id* değeri *paragraf1* olan öğeye erişerek yazı rengini maviye dönüştürür.

getElementsByClassName()

Tanım: Aynı sınıf (class) değerine sahip birden fazla öğeyi seçmek için kullanılır.

Özellik: Bir HTMLCollection döndürür. Yani birden fazla öğeye erişim sağlanır.

Örnek:

```
<p class="bilgi">Paragraf 1</p>
<p class="bilgi">Paragraf 2</p>
<script>
  let paragraflar = document.getElementsByClassName("bilgi");
  for(let i = 0; i < paragraflar.length; i++) {
    paragraflar[i].style.fontWeight = "bold";
  }
</script>
```

Burada aynı sınıfa sahip tüm paragraflar **kalin (bold)** hale getirilmiştir.

getElementsByTagName()

Tanım: Belirli bir HTML etiketine (örneğin <p>, <div>, <h1>) sahip tüm öğeleri seçmek için kullanılır.

Özellik: HTMLCollection döndürür ve genellikle döngüyle birlikte kullanılır.

Örnek:

```
<p>Birinci paragraf</p>
<p>İkinci paragraf</p>
<script>
  let tumParagraflar = document.getElementsByTagName("p");
  for(let i = 0; i < tumParagraflar.length; i++) {
    tumParagraflar[i].style.backgroundColor = "yellow";
  }
</script>
```

Tüm <p> etiketlerinin arka plan rengi sarıya boyanmıştır.

querySelector()

Tanım: CSS seçicileri kullanarak DOM'daki ilk eşleşen öğeyi seçmek için kullanılır.

Özellik: Yalnızca ilk öğeyi döndürür.

Örnek:

```
<div class="icerik">İçerik 1</div>
<div class="icerik">İçerik 2</div>
<script>
  let div = document.querySelector(".icerik");
  div.style.border = "1px solid red";
</script>
```

Bu kod yalnızca ilk *.icerik* sınıfına sahip div ögesini etkiler.

querySelectorAll()

Tanım: CSS seçicilerini kullanarak tüm eşleşen öğeleri seçer.

Özellik: Bir NodeList döndürür, dolayısıyla `forEach` ile kolayca döngü kurulabilir.

Örnek:

```
<div class="kutu">Kutu 1</div>
<div class="kutu">Kutu 2</div>
<script>
  let kutular = document.querySelectorAll(".kutu");
  kutular.forEach(function(k) {
    k.style.padding = "10px";
  });
</script>
```

Burada tüm. kutu öğelerine dolgu (padding) eklenmiştir.

document.getElementsByName()

Tanım: Özellikle formlarda kullanılan *name* niteliğine göre öğeleri seçer.

Örnek:

```
<input type="text" name="kullanici">
```

```
<input type="text" name="kullanici">

<script>
  let alanlar = document.getElementsByName("kullanici");
  alanlar.forEach(a => a.value = "Varsayılan");
</script>
```

Bu kod, tüm *name="kullanici"* alanlarına "Varsayılan" değerini atar.

Tablo 9.1. DOM'da Ögelere Erişim Yöntemlerinin Karşılaştırılması

Yöntem	Döndürdüğü Tür	Kullanım Senaryosu
getElementById	Tek öge	Belirli ve eşsiz id seçimi
getElementsByClassName	HTMLCollection	Aynı sınıfa sahip çoklu ögeler
getElementsByTagName	HTMLCollection	Belirli etiket türündeki ögeler
querySelector	Tek öge	CSS seçicisi ile ilk eşleşme
querySelectorAll	NodeList	CSS seçicisi ile tüm eşleşmeler
getElementsByName	NodeList	Form alanlarında <i>name</i> niteliği ile seçim

DOM erişim yöntemlerinin çeşitliliği, geliştiricilere esneklik sağlar. Örneğin, tekil bir ögeye erişmek için `getElementById` en verimli yöntemdir. Ancak karmaşık seçimler gerektiğinde `querySelector` veya `querySelectorAll` daha esnek bir çözüm sunar. Etkin web geliştirme için bu yöntemlerin tümünün bilinmesi ve uygun bağlamda kullanılması önemlidir.

DOM Üzerinde İçerik ve Stil Değişiklikleri

DOM'a erişim sağlandıktan sonra geliştiricinin en sık başvurduğu işlemlerden biri, mevcut ögelerin içeriklerini ve görünümünü değiştirmektir. Web sayfalarının dinamik olmasının en temel sebebi de bu esnekliktir. JavaScript, DOM aracılığıyla hem öge içeriklerini (text, HTML) hem de öge stillerini (CSS özellikleri) programatik olarak değiştirmeye olanak tanır.

İçerik Değiştirme

Bir HTML ögesinin içerisindeki metni veya HTML etiketlerini değiştirmek için birkaç farklı özellik kullanılabilir.

a. innerHTML

- Ögenin içeriğini HTML biçiminde değiştirmek için kullanılır.
- Yalnızca metin değil, aynı zamanda HTML etiketlerini de işler.

```
<p id="metin">Merhaba</p>
<script>
  document.getElementById("metin").innerHTML = "<b>Merhaba
Dünya</b>";
</script>
```

Sonuç olarak paragraf içerisindeki yazı kalın hale gelir.

b. innerText

- Ögenin yalnızca metin içeriğini değiştirir.
- HTML etiketlerini işleme almaz.

```
<p id="metin2">Merhaba</p>
<script>
    document.getElementById("metin2").innerText = "<b>Merhaba
Dünya</b>";
</script>
```

Bu durumda paragrafın içinde **Merhaba Dünya** yazısı düz metin olarak görünür.

c. textContent

- innerText'e benzer şekilde yalnızca metin değerini döndürür veya değiştirir.
- Ancak innerText tarayıcıda stil (örneğin display: none) dikkate alırken, textContent tüm metni getirir.

Öznitelik (Attribute) Değiştirme

Bir HTML ögesinin öznitelikleri (src, href, alt, title vb.) JavaScript ile dinamik olarak güncellenebilir.

a. setAttribute() ve getAttribute()

```

<script>
    let img = document.getElementById("resim");
    img.setAttribute("src", "yeni.jpg");
    console.log(img.getAttribute("alt"));
</script>
```

Bu örnekte resmin kaynağı değiştirilmiş, ayrıca alt özniteliği konsola yazdırılmıştır.

b. Doğrudan özellik kullanımı

```
document.getElementById("resim").src = "yeniresim.jpg";
```

Stil (CSS) Değiştirme

DOM manipülasyonunda CSS özelliklerini değiştirmek oldukça yaygındır.

a. style özelliği ile inline stil değiştirme

```
<p id="yazi">Stil örneği</p>
<script>
    let y = document.getElementById("yazi");
    y.style.color = "red";
    y.style.fontSize = "20px";
</script>
```

Bu kod yazının rengini kırmızıya, boyutunu 20 piksele dönüştürür.

b. CSS sınıflarını değiştirme (classList)

Bir öğeye stil vermenin en etkili yollarından biri sınıf (class) tanımları üzerinden ilerlemektir.

- `classList.add("...")` → Yeni bir sınıf ekler.
- `classList.remove("...")` → Var olan sınıfı siler.
- `classList.toggle("...")` → Sınıf varsa kaldırır, yoksa ekler.

```
<style>
  .vurgulu { color: blue; font-weight: bold; }
</style>
<p id="paragraf">Bu bir paragraf</p>
<script>
  let p = document.getElementById("paragraf");
  p.classList.add("vurgulu");
</script>
```

Sonuçta paragraf mavi ve kalın hale gelir.

Dinamik Stil Uygulamaları

DOM manipülasyonu ile yapılan içerik ve stil değişiklikleri, web sitelerinin daha canlı bir görünüme kavuşmasını sağlar. Örneğin:

- **Tema Değiştirme:** Kullanıcı tek bir butona tıklayarak “dark mode” veya “light mode” arasında geçiş yapabilir.
- **Form Geri Bildirimi:** Kullanıcı doğru veri girdiğinde alan yeşil, yanlış girdiğinde kırmızı renkle vurgulanabilir.
- **Animasyon Tetikleme:** Bir öğeye tıklandığında CSS animasyonu başlatılabilir.

Örnek: Tema Değiştirme Uygulaması

```
<style>
  body.light { background: white; color: black; }
  body.dark { background: black; color: white; }
</style>
<button id="buton">Tema Değiştir</button>
<script>
  let b = document.getElementById("buton");
  b.addEventListener("click", function() {
    document.body.classList.toggle("dark");
    document.body.classList.toggle("light");
  });
</script>
```

Bu kod sayesinde kullanıcı her tıklamada tema değiştirebilir.

Yeni Öğeler Ekleme ve Silme İşlemleri

Web sayfalarının etkileşimli ve dinamik olmasını sağlayan bir diğer önemli unsur, kullanıcı eylemlerine veya belirli koşullara bağlı olarak yeni içeriklerin eklenebilmesi ve mevcut içeriklerin kaldırılabilmesidir. DOM manipülasyonu, yalnızca var olan içerikleri değiştirmekle kalmaz; aynı zamanda sayfaya tamamen

yeni öğeler ekleme veya gereksiz öğeleri kaldırma olanağı da sunar. Bu işlev, modern web uygulamalarının temelini oluşturur. Örneğin, sosyal medya platformlarında yeni bir gönderi oluşturulduğunda anında sayfaya eklenmesi veya bir alışveriş sepetinde ürünlerin silinmesi bu işlemler sayesinde gerçekleştirilir.

Yeni Öğeler Oluşturma

JavaScript ile DOM'a yeni bir öğe eklemek için öncelikle öğe oluşturulmalı, ardından uygun bir yere eklenmelidir. Bu işlem genellikle üç adımdan oluşur:

- `document.createElement("etiket")` ile *yeni öğe* oluşturulur.
- Gerekirse `textContent` veya `innerHTML` ile içeriği belirlenir.
- `appendChild()` veya `append()` gibi yöntemlerle sayfaya eklenir.

Örnek: Listeye Yeni Öğeler Ekleme

```
<ul id="liste">
  <li>Öge 1</li>
</ul>
<script>
  let yeniOge = document.createElement("li"); // 1. Ögeyi
oluştur
  yeniOge.textContent = "Öge 2"; // 2. içeriğini
belirle
  document.getElementById("liste").appendChild(yeniOge); //
3. listeye ekle
</script>
```

Sonuç olarak listeye dinamik biçimde yeni bir madde eklenir.

Belirli Bir Konuma Ekleme

Bazı durumlarda yeni öğe listenin en sonuna değil, belirli bir konumuna eklenmek istenebilir. Bunun için `insertBefore()` yöntemi kullanılabilir.

```
<ul id="liste2">
  <li>Birinci</li>
  <li>Üçüncü</li>
</ul>
<script>
  let ikinci = document.createElement("li");
  ikinci.textContent = "İkinci";
  let liste = document.getElementById("liste2");
  liste.insertBefore(ikinci, liste.children[1]);
</script>
```

Bu örnekte "İkinci" ögesi, "Birinci" ve "Üçüncü" arasında yerleştirilmiştir.

Ögeleri Silme

DOM'dan öğe kaldırmak için birkaç yöntem kullanılabilir.

a. `removeChild()`

Bir ebeveyn öğe içerisinden belirli bir çocuk öğeyi kaldırır.

```
<ul id="liste3">
  <li>Silinecek</li>
  <li>Kalsın</li>
</ul>
<script>
  let liste = document.getElementById("liste3");
  let silinecek = liste.children[0];
  liste.removeChild(silinecek);
</script>
```

b. `remove()`

Doğrudan seçilen öğeyi kaldırır.

```
<p id="paragraf">Bu paragraf silinecek.</p>
<script>
  document.getElementById("paragraf").remove();
</script>
```

Daha İleri Kullanımlar

- **Form Alanı Dinamik Ekleme:** Kullanıcı "Alan Ekle" butonuna bastığında yeni input kutusu eklemek.
- **Dinamik Tablo Satırı:** Tabloya yeni bir satır ekleme veya mevcut satırı kaldırma.
- **İçerik Akışı:** Sosyal medya uygulamalarında yeni gönderilerin eklenmesi.

Örnek: Dinamik Görev Listesi

```
<input type="text" id="gorev" placeholder="Görev yazın">
<button id="ekle">Ekle</button>
<ul id="gorevListesi"></ul>
<script>
  let ekleButon = document.getElementById("ekle");
  ekleButon.addEventListener("click", function() {
    let gorevMetni =
document.getElementById("gorev").value;
    if(gorevMetni !== "") {
      let yeni = document.createElement("li");
      yeni.textContent = gorevMetni;
      // silme butonu
      let sil = document.createElement("button");
      sil.textContent = "Sil";
      sil.addEventListener("click", function() {
        yeni.remove();
      });
      yeni.appendChild(sil);
    }
  });
  yeni.appendChild(sil);
```

```

        document.getElementById("gorevListesi").appendChild
(yeni);
    }
});
</script>

```

Bu örnekte kullanıcı bir görev yazıp "Ekle" butonuna bastığında görev listeye eklenir. Her görevin yanında "Sil" butonu bulunur ve bu butonla görev kolayca kaldırılabilir.

Yeni öge ekleme ve silme işlemleri, web sayfalarının statik bir yapıdan çıkıp kullanıcı odaklı bir platforma dönüşmesini sağlar. Bu yöntemlerle geliştiriciler, kullanıcı etkileşimlerine anlık yanıt veren, dinamik ve esnek uygulamalar geliştirebilirler. Özellikle to-do listeler, alışveriş sepetleri, sosyal medya içerik akışları gibi günlük yaşamda sık karşılaşılan uygulamaların temelinde bu teknikler yer almaktadır.

Örnek Uygulama: Dinamik İçerik Güncelleme

DOM manipülasyonu, teorik olarak öğelere erişim, içerik ve stil değişikliği ile yeni öge ekleme/silme işlemlerini kapsar. Ancak bu tekniklerin gerçek gücü, bir arada kullanıldığında ortaya çıkar. Bu konunun içselleştirebilmesi için dinamik içerik güncelleme uygulamaları kritik bir öğrenme basamağıdır. Bu tür uygulamalar sayesinde web sayfaları, kullanıcı girişlerine veya belirli koşullara göre kendini yenileyebilir.

Örnek Senaryo: Haber Duyuruları Paneli

Bir web sitesinde yöneticiler yeni bir duyuru eklediğinde, bu duyurunun anında kullanıcıya yansıtılması gerekir. Aynı şekilde eski bir duyuru kaldırıldığında bu da sayfa üzerinden silinmelidir. Bu senaryoyu küçük bir uygulama ile gösterebiliriz.

```

<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <title>DuyuruPaneli</title>
  <style>
    body { font-family: Arial, sans-serif; }
    #duyurular { margin-top: 20px; }
    .duyuru {
      padding: 10px;
      margin: 5px 0;
      background: #f1f1f1;
      border-radius: 5px;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    .sil {

```

```
        background: red;
        color: white;
        border: none;
        padding: 5px 10px;
        cursor: pointer;
        border-radius: 3px;
    }
</style>
</head>
<body>
    <h1>Duyuru Paneli</h1>
    <input type="text" id="yeniDuyuru" placeholder="Yeni duyuru
    girin">
    <button id="ekle">Ekle</button>
    <div id="duyurular"></div>
    <script>
        const ekleButon = document.getElementById("ekle");
        const duyurular = document.getElementById("duyurular");
        ekleButon.addEventListener("click", function() {
            const metin = document.getElementById("yeniDuyuru").value;
            if(metin !== "") {
                // yeni duyuru div'i oluştur
                const duyuru = document.createElement("div");
                duyuru.classList.add("duyuru");
                duyuru.textContent = metin;
                // sil butonu ekle
                const silButon = document.createElement("button");
                silButon.textContent = "Sil";
                silButon.classList.add("sil");
                // silme işlemi
                silButon.addEventListener("click", function() {
                    duyuru.remove();
                });
                duyuru.appendChild(silButon);
                duyurular.appendChild(duyuru);
                // inputu temizle
                document.getElementById("yeniDuyuru").value = "";
            }
        });
    </script>
</body>
</html>
```

Uygulamanın Açıklaması

- **Öge ekleme:** Kullanıcı bir metin yazıp "Ekle" butonuna bastığında yeni bir div oluşturulmakta ve duyurular listesine eklenmektedir.
- **Stil uygulama:** CSS ile her duyuru kutusu arka plan, kenar yuvarlama ve aralıklarla biçimlendirilmiştir.

- **Öge silme:** Her duyurunun yanında "Sil" butonu yer almakta; bu butona basıldığında duyuru DOM'dan kaldırılmaktadır.
- **Dinamik deneyim:** Sayfa yenilenmeden içerik güncellenmekte, kullanıcıya etkileşimli bir deneyim sunulmaktadır.

Gerçek Hayat Bağlantısı

Bu uygulama küçük bir örnek olmasına rağmen, modern web sitelerinde çok sık karşılaşılan bir işlevselliği temsil eder. Haber sitelerindeki son dakika kutuları, sosyal medya platformlarındaki gönderi akışları veya e-ticaret sitelerindeki sepet güncellemeleri aynı mantıkla çalışır. Dolayısıyla bu örnek üzerinden, DOM manipülasyonunun günlük web deneyimindeki önemi kavranabilir.

OLAY YÖNETİMİ (EVENT HANDLING)

Web tasarımında yalnızca içerik ve stil değişiklikleri yeterli değildir; asıl önemli olan, kullanıcı ile web sayfası arasında etkileşimli bir iletişim kurabilmektir. Bir kullanıcının fare ile bir butona tıklaması, klavyeden bir tuşa basması, bir form alanına veri girmesi veya fareyi belirli bir ögenin üzerine getirmesi web tarayıcıları tarafından "*olay (event)*" olarak tanımlanır. Bu olayların yakalanması ve bunlara uygun tepkiler verilmesi süreci ise *event handling (olay yönetimi)* olarak adlandırılır.

JavaScript'in en güçlü yönlerinden biri, bu olayları dinleyebilme ve onlara anında tepki verebilme yeteneğidir. Olay yönetimi sayesinde geliştirici, kullanıcı eylemlerine uygun davranışlar tanımlayarak sayfanın dinamik bir yapıya kavuşmasını sağlar. Örneğin, bir "*Gönder*" butonuna basıldığında formun doğrulanması, bir resmin üzerine gelindiğinde büyütülmesi ya da klavyeden belirli bir tuşa basıldığında özel bir fonksiyonun çalıştırılması event handling ile mümkün hale gelir.

Olay yönetimi aynı zamanda kullanıcı deneyimini (UX) doğrudan etkileyen bir unsurdur. Çünkü web sayfalarının kullanıcıya verdiği tepkiler, sistemin ne kadar duyarlı ve işlevsel olduğunu belirler. Bu nedenle olayların doğru yakalanması, performanslı bir şekilde yönetilmesi ve kullanıcı dostu geri bildirimlerle desteklenmesi modern web tasarımının vazgeçilmez ilkelerindendir.

Bu bölümde öncelikle olay kavramı ve türleri ele alınacak, ardından JavaScript'te *olay dinleyicilerinin (event listeners)* nasıl tanımlandığı ve yönetildiği açıklanacaktır. Sonrasında en sık kullanılan olay türleri üzerinde durulacak, event bubbling ve capturing gibi ileri düzey kavramlara giriş yapılacaktır. Bölümün sonunda ise çeşitli örnek uygulamalar ile olay yönetimi pratik olarak gösterilecektir.

OLAY KAVRAMI VE TÜRLERİ

Olay (Event) Nedir?

Web sayfalarında kullanıcı ile tarayıcı arasındaki etkileşimi temsil eden en temel mekanizma olay (event) kavramıdır. Bir olay, kullanıcının gerçekleştirdiği

eylemler (örneğin fare tıklaması, klavyeden tuşa basılması, form alanına veri girilmesi) veya tarayıcı tarafından tetiklenen durumlar (sayfanın yüklenmesi, hata oluşması, medya dosyasının oynatılması) sonucunda meydana gelir. Bu olayların yakalanarak belirli fonksiyonlarla işlenmesi, web sayfalarının dinamik ve etkileşimli hale gelmesini sağlar.

JavaScript, olayları yakalamak ve bunlara tepki vermek için güçlü bir altyapı sunar. Böylece geliştirici, “kullanıcı şu işlemi yaptığında, sayfada şu değişiklik gerçekleşsin” şeklinde koşullar tanımlayabilir. Bu, kullanıcı deneyiminin zenginleşmesine ve web sitelerinin sadece okunabilir içerikten çıkıp etkileşimli uygulamalar düzeyine yükselmesine katkıda bulunur.

Olay Türleri

Olaylar, kullanıcı veya sistem tarafından tetiklenme biçimlerine göre farklı kategorilere ayrılır. Aşağıda en sık karşılaşılan olay türleri ve kullanım alanları özetlenmiştir:

1. Fare (Mouse) Olayları

Kullanıcının fare ile gerçekleştirdiği hareketleri temsil eder.

- *click*: Ögeye tıklama
- *dblclick*: Çift tıklama
- *mouseover*: Fare ile ögenin üzerine gelme
- *mouseout*: Fare ile ögeden ayrılma
- *mousemove*: Fare hareket ettirildiğinde tetiklenir

Örnek:

```
document.getElementById("buton").addEventListener("click",  
function() {  
    alert("Butona tıklandı!");  
});
```

2. Klavye (Keyboard) Olayları

Kullanıcının klavyede yaptığı eylemleri yakalar.

- *keydown*: Bir tuşa basıldığında tetiklenir
- *keypress*: Tuş basılı tutulduğunda sürekli tetiklenir (güncel tarayıcılarda daha az kullanılır)
- *keyup*: Tuş bırakıldığında tetiklenir

Örnek:

```
document.addEventListener("keydown", function(event) {  
    console.log("Basılan tuş:", event.key);  
});
```

3. Form Olayları

Form öğeleri üzerinde kullanıcı girişlerini kontrol etmek için kullanılır.

- *submit*: Form gönderildiğinde
- *change*: Bir alanın değeri değiştiğinde
- *input*: Kullanıcı yazı yazarken anlık olarak tetiklenir
- *focus*: Bir alan seçildiğinde (ör. kullanıcı metin kutusuna tıkladığında)
- *blur*: Odak (focus) kaybolduğunda

Örnek:

```
document.getElementById("ad").addEventListener("blur", function() {  
    alert("Alan boş bırakılamaz!");  
});
```

4. Pencere (Window) Olayları

Tarayıcı penceresinin durumuna göre tetiklenir.

- **load**: Sayfa tamamen yüklendiğinde
- **resize**: Pencere boyutu değiştiğinde
- **scroll**: Sayfa kaydırıldığında
- **unload**: Sayfa kapatılmadan önce

Örnek:

```
window.addEventListener("resize", function() {  
    console.log("Pencere boyutu değişti");  
});
```

5. Medya Olayları

Ses veya video gibi medya dosyalarının durumunu izlemek için kullanılır.

- **play**: Medya oynatıldığında
- **pause**: Medya durdurulduğunda
- **ended**: Medya bittiğinde

Örnek:

```
let video = document.getElementById("video");  
video.addEventListener("ended", function() {  
    alert("Video tamamlandı");  
});
```

Olay Türlerinin Önemi

Olay türlerinin çeşitliliği, geliştiricilerin kullanıcı davranışlarını ayrıntılı olarak yakalamasına olanak tanır. Bu sayede:

- Kullanıcı deneyimi geliştirilebilir,
- Form girişleri anında kontrol edilebilir,
- Dinamik görsel efektler uygulanabilir,
- Sayfanın farklı koşullara adaptasyonu sağlanabilir (örneğin mobil uyumlu tasarımlarda resize olayı).

Olay Dinleyicileri (addEventListener) Kullanımı**Olay dinleyicisi nedir?**

Bir web sayfasında kullanıcı veya sistem tarafından tetiklenen olayların işlenebilmesi için bu olayların bir fonksiyon ile ilişkilendirilmesi gerekir. Bu ilişkilendirme sürecinde kullanılan mekanizmaya olay dinleyicisi (event listener) adı verilir. Olay dinleyicileri, belirli bir olay gerçekleştiğinde çalışacak kodu belirleyen yapılardır.

JavaScript'te olay dinleyicileri farklı yöntemlerle tanımlanabilir. En yaygın kullanılan yöntemlerden biri, HTML etiketlerine doğrudan olay özellikleri eklemek iken, modern yaklaşım `addEventListener()` metodunu kullanmaktır.

1. Geleneksel Yöntem: HTML İçinde Olay Tanımlama

Eski ama hâlâ kullanılabilen yöntem, olayın doğrudan HTML etiketi içinde tanımlanmasıdır.

```
<button onclick="alert('Butona tıklandı!')">Tıkla</button>
```

Bu yaklaşım basit durumlarda işe yarasa da kodun okunabilirliğini düşürür ve HTML ile JavaScript'in birbirine karışmasına yol açar. Bu nedenle güncel web geliştirme pratiklerinde tercih edilmemektedir.

2. Modern Yöntem: `addEventListener()`

`addEventListener()` metodu, DOM API tarafından sağlanan ve modern web geliştirme süreçlerinde yaygın olarak kullanılan bir yöntemdir. Bu metodun temel avantajları şunlardır:

- Bir öğeye birden fazla olay dinleyicisi atanabilir.
- HTML ve JavaScript ayrı tutulur, böylece kodun bakımı kolaylaşır.
- Olay dinleyicileri istendiğinde kaldırılabilir (`removeEventListener`).

Temel Söz Dizimi

```
element.addEventListener(olayTürü, fonksiyon, seçenekler);
```

- **Element:** Olayın atanacağı DOM ögesi.
- **Olay Türü:** "click", "mouseover", "keydown" gibi olayın adı.
- **Fonksiyon:** Olay gerçekleştiğinde çalışacak fonksiyon.
- **Seçenekler (opsiyonel):** Olayın yakalanma aşaması veya yalnızca bir kez çalışması gibi ek ayarlar.

3. Kullanım Örnekleri

a. Butona tıklama olayı

```
<button id="buton">Tıkla</button>
<script>
  let b = document.getElementById("buton");
  b.addEventListener("click", function() {
    alert("Butona tıklandı!");
  });
</script>
```

b. Fare ile üzerine gelme

```
<script>
  let kutu = document.getElementById("kutu");
  kutu.addEventListener("mouseover", function() {
    kutu.style.backgroundColor = "lightblue"; });
</script>
```

c. Klavyeden tuşa basma

```
<script>
  document.addEventListener("keydown", function(event) {
    console.log("Basılan tuş:", event.key);
  });
</script>
```

4. Birden Fazla Dinleyici Atama

Aynı öğeye *birden fazla dinleyici* eklenebilir.

```
<script>
  let btn = document.getElementById("btn");
  btn.addEventListener("click", function() {
    console.log("Birinci dinleyici çalıştı");
  });
  btn.addEventListener("click", function() {
    console.log("İkinci dinleyici çalıştı");
  });
</script>
```

Kullanıcı butona tıkladığında *iki fonksiyon* da çalışır.

5. Olay Dinleyicisini Kaldırma

Bir olay dinleyicisi daha sonra kaldırılmak istenirse `removeEventListener()` kullanılabilir. Bunun için fonksiyonun bir değişkene atanmış olması gerekir.

```
<script>
  function mesaj() {
    alert("Tıklandı!");
  }
  let b = document.getElementById("buton");
  b.addEventListener("click", mesaj);
  // kaldırmak için
  b.removeEventListener("click", mesaj);
</script>
```

6. Opsiyonel Parametreler

- { once: true } → Olay dinleyicisi yalnızca bir kez çalışır.
- { capture: true } → Olay yakalamanın capturing aşamasında çalışmasını sağlar.

```
btn.addEventListener("click", () => alert("Sadece bir kez çalışır"), {
  once: true });
```

`addEventListener()` metodu, web geliştirme sürecinde olay yönetimini daha esnek, ölçeklenebilir ve modüler hale getiren bir yaklaşımdır. Bu yöntemin etkin şekilde kullanılmasının öğrenilmesi hem temiz kod yazma becerisi hem de büyük projelerde karmaşıklığı azaltma açısından kritik öneme sahiptir.

Sık Kullanılan Olaylar

Olay dinleyicisi nedir?

JavaScript ile olay yönetimi yapılırken yüzlerce farklı olay türü kullanılabilir. Ancak web geliştirme sürecinde bazı olaylar diğerlerinden çok daha sık karşımıza

çıkartır. Bu olaylar, kullanıcı ile web sayfası arasındaki temel etkileşimleri temsil eder ve dinamik davranışların büyük bir kısmını oluşturur.

Aşağıda en sık kullanılan olay türleri ve bunların örnek uygulamaları ele alınmaktadır.

1. click Olayı

En yaygın kullanılan olay türlerinden biridir. Kullanıcı bir öğeye tıkladığında tetiklenir.

```
<button id="buton">Tıkla</button>
<script>
    document.getElementById("buton").addEventListener("click",
function() {
    alert("Butona tıklandı!");
});
</script>
```

Uygulama Alanları:

- Buton tıklamaları
- Menülerin açılıp kapanması
- Dinamik içerik yükleme

2. dblclick Olayı

Kullanıcının öğeye *çift tıklamasıyla* tetiklenir.

```
<p id="yazi">Bu metni çift tıklayın.</p>
<script>
    document.getElementById("yazi").addEventListener("dblclick"
, function() {
    this.style.color = "red";
});
</script>
```

Uygulama Alanları:

- Metin düzenleme başlatma
- Özel etkileşimli öğeler

3. mouseover ve mouseout Olayları

Kullanıcı fareyi bir öğenin üzerine getirdiğinde (mouseover) veya üzerinden geçtiğinde (mouseout) tetiklenir.

```
<div id="kutu" style="width:100px; height:100px;
background:lightgray;"></div>
<script>
    let kutu = document.getElementById("kutu");
    kutu.addEventListener("mouseover", () =>
kutu.style.background = "green");
    kutu.addEventListener("mouseout", () =>
kutu.style.background = "lightgray");
</script>
```

Uygulama Alanları:

- Menü öğelerinde hover efektleri
- Görsellerin önizlemesi
- Dinamik vurgulama

4. input Olayı

Form alanına yazı yazıldığında anlık olarak tetiklenir.

```
<input type="text" id="ad" placeholder="Adınızı girin">
<p id="mesaj"></p>
<script>
  let ad = document.getElementById("ad");
  let mesaj = document.getElementById("mesaj");
  ad.addEventListener("input", function() {
    mesaj.textContent = "Merhaba, " + ad.value;
  });
</script>
```

Uygulama Alanları:

- Anlık arama önerileri
- Form doğrulama
- Kullanıcıya canlı geri bildirim

5. keydown, keyup ve keypress Olayları

Kullanıcı klavyeden bir tuşa bastığında (keydown), basılı tutarken (keypress) veya tuşu bıraktığında (keyup) tetiklenir.

```
<input type="text" id="klavye" placeholder="Bir şey yazın">
<script>
  let k = document.getElementById("klavye");
  k.addEventListener("keydown", (event) => console.log("Basılan tuş:", event.key));
  k.addEventListener("keyup", (event) => console.log("Bırakılan tuş:", event.key));
</script>
```

Uygulama Alanları:

- Kısayol tuş kombinasyonları
- Oyun kontrolleri
- Anlık yazım doğrulaması

6. submit Olayı

Bir form gönderildiğinde tetiklenir.

```
<form id="form">
  <input type="text" placeholder="Kullanıcı adı">
  <button type="submit">Gönder</button>
</form>
<script>
  document.getElementById("form").addEventListener("submit", function(event) {
```

```
event.preventDefault(); // sayfanın yenilenmesini engeller
alert("Form gönderildi!");
});
</script>
```

Uygulama Alanları:

- Form doğrulama
- Sunucuya veri gönderme
- AJAX tabanlı veri işleme

7. change Olayı

Bir form alanındaki değer değiştirildiğinde tetiklenir (özellikle *select* kutuları için kullanışlıdır).

```
<select id="secim">
  <option>Elma</option>
  <option>Armut</option>
  <option>Muz</option>
</select>
<script>
  document.getElementById("secim").addEventListener("change",
function() {
  alert("Seçilen: " + this.value);
});
</script>
```

8. scroll Olayı

Kullanıcı sayfayı kaydırıldığında tetiklenir.

```
<script>
  window.addEventListener("scroll", function() {
    console.log("Sayfa kaydırılıyor: ", window.scrollY);
  });
</script>
```

Uygulama Alanları:

- Sonsuz kaydırma (infinite scroll)
- “Yukarı Çık” butonunun görünmesi
- Animasyonların tetiklenmesi

Event Bubbling ve Capturing Kavramlarına Giriş

Olayların Yayılımı (Event Propagation)

Bir web sayfasında olaylar yalnızca hedeflenen öğede gerçekleşmez; DOM’un hiyerarşik yapısı gereği olaylar farklı aşamalardan geçerek *yayılım* (*propagation*) süreci oluşturur. Yani, kullanıcı bir öğe üzerinde bir eylem gerçekleştirdiğinde bu olay yalnızca o öğede kalmaz, aynı zamanda öğenin ebeveynlerine ve en sonunda *kök* (*root*) öğeye kadar yayılabilir.

JavaScript’te olay yayılımı üç aşamadan oluşur:

1. Capturing Phase (Yakalama Aşaması)
2. Target Phase (Hedef Aşaması)

3. Bubbling Phase (Kabarcıklanma Aşaması)

1. Capturing (Yakalama Aşaması)

Olay, DOM ağacının en üstündeki ögeden (örneğin; document veya window) başlar ve hedef ögeye doğru aşağıya iner. Bu aşamada, olay “yukarıdan aşağıya” ilerler. Eğer olay dinleyicisi { capture: true } parametresi ile atanmışsa, olay yakalama aşamasında çalıştırılabilir.

```
<script>
  document.getElementById("disDiv").addEventListener("click",
function() {
  console.log("Dış div - capturing");
}, true);
</script>
```

2. Target (Hedef Aşaması)

Olayın doğrudan gerçekleştiği ögedir. Örneğin, kullanıcı bir butona tıklamışsa bu aşamada olay doğrudan o butonda işlenir.

3. Bubbling (Kabarcıklanma Aşaması)

Olay, hedef ögeden başlayarak DOM ağacında yukarı doğru “baloncuk” gibi yükselir. Yani tıklanan ögenin ebeveynlerine, daha üst kapsayıcı öğelere ve en sonunda document seviyesine kadar ulaşır. Varsayılan olarak, JavaScript olay dinleyicileri bubbling aşamasında çalışır.

```
<div id="disDiv" style="padding:30px; background:lightblue;">
  Dış Div
  <button id="buton">Tıkla</button>
</div>
<script>
  document.getElementById("disDiv").addEventListener("click",
function() {
  console.log("Dış div - bubbling");
});
  document.getElementById("buton").addEventListener("click",
function() {
  console.log("Buton");
});
</script>
```

Butona tıklandığında önce “Buton” ardından “Dış div - bubbling” mesajları görülür.

4. Olay Yayılımını Durdurma

Bazı durumlarda olayın üst öğelere yayılmasını engellemek gerekebilir. Bunun için event.stopPropagation() metodu kullanılır.

```
<script>
  document.getElementById("buton").addEventListener("click",
function(event) {
  console.log("Butona tıklandı");
  event.stopPropagation(); // olay dış div'e gitmez
});
</script>
```

```
});  
</script>
```

Bu durumda butona tıklandığında yalnızca butonun dinleyicisi çalışır, dış div etkilenmez.

5. Olay Önceliğini Yönetme

Olayın bubbling mi yoksa capturing aşamasında mı çalışacağını kontrol etmek için addEventListener metoduna üçüncü parametre olarak { capture: true } verilmesi gerekir.

```
<script>  
  disDiv.addEventListener("click", function() {  
    console.log("Dış div - capturing");  
  }, true);  
</script>
```

Bu durumda dış div'in olayı capturing aşamasında çalışır ve butonun olayından önce tetiklenir.

6. Kullanım Alanları

Bubbling:

- Genellikle olayların yukarıya doğru taşınması için tercih edilir.
- Örneğin bir tablo içerisindeki satırlara tıklama olayını, her hücreye ayrı ayrı eklemek yerine sadece tabloya eklemek mümkündür.

Capturing:

- Daha özel kontrol mekanizmaları gerektiğinde veya olayın daha alt öğelere ulaşmadan işlenmesi istendiğinde kullanılır.

Örnek Uygulama: Tıklama ve Klavye Etkileşimleri

Olay yönetimi (event handling) kavramını en iyi şekilde anlamanın yolu, teorik bilgiyi gerçek uygulamalara dönüştürmektir. Bu bölümde hem fare tıklaması (click) hem de klavye etkileşimleri (keydown, keyup) bir arada kullanılarak dinamik bir web sayfası hazırlanacaktır.

Amaç, kullanıcının etkileşimleri sonucunda anlık olarak değişebilen bir içerik sunmak ve olayların nasıl yakalanıp işlendiğini göstermektir.

Senaryo: Basit Bir Etkileşimli Not Defteri

Kullanıcı, metin kutusuna bir şey yazdığında klavye olaylarıyla bu giriş yakalanacak, "Ekle" butonuna tıkladığında ise metin bir listeye eklenecektir. Ayrıca liste öğeleri tıklandığında silinebilecek ve kullanıcıya geri bildirim verilecektir.

Uygulama kodu

```
<!DOCTYPE html>  
<html lang="tr">  
<head>  
  <meta charset="UTF-8">  
  <title>Etkileşimli Not Defteri</title>  
<style>
```

```
body { font-family: Arial, sans-serif; margin: 30px; }
#notlar { margin-top: 20px; }
li {
  background: #f1f1f1;
  margin: 5px 0;
  padding: 8px;
  border-radius: 5px;
  cursor: pointer;
}
li:hover { background: #d1f7d1; }
#bilgi { color: gray; margin-top: 10px; }
</style>
</head>
<body>
  <h1>Etkileşimli Not Defteri</h1>
  <input type="text" id="notInput" placeholder="Notunuzu yazın">
  <button id="ekle">Ekle</button>
  <p id="bilgi">Not yazıp Enter'a basabilir veya "Ekle" butonunu
kullanabilirsiniz.</p>
  <ul id="notlar"></ul>
  <script>
    const input = document.getElementById("notInput");
    const ekleBtn = document.getElementById("ekle");
    const liste = document.getElementById("notlar");
    const bilgi = document.getElementById("bilgi");
    // Klavyeden Enter tuşu ile ekleme
    input.addEventListener("keydown", function(event) {
      if(event.key === "Enter") {
        notEkle();
      }
    });
    // Butona tıklayarak ekleme
    ekleBtn.addEventListener("click", notEkle);
    function notEkle() {
      const metin = input.value.trim();
      if(metin !== "") {
        let li = document.createElement("li");
        li.textContent = metin;
        // Notu silmek için tıklama olayı
        li.addEventListener("click", function() {
          li.remove();
          bilgi.textContent = `${metin}` notu silindi.`;
        });
        liste.appendChild(li);
        input.value = "";
        bilgi.textContent = `${metin}` notu eklendi.`;
      }
    }
  </script>
```



```
</body>  
</html>
```

Uygulamanın Açıklaması

1. Klavye Etkileşimi (keydown):

Kullanıcı metin kutusuna yazı yazarken *Enter* tuşuna bastığında yeni not listeye eklenmektedir.

2. Tıklama Etkileşimi (click):

- "Ekle" butonuna tıklanarak not eklenebilir.
- Eklenen her liste ögesi (not), üzerine tıklandığında silinir.

3. Dinamik Geri Bildirim:

Kullanıcıya her işlemten sonra bilgi mesajı gösterilir. Bu, olay yönetiminin kullanıcı deneyimini geliştirmede ne kadar önemli olduğunu gösterir.

Gerçek Hayat Bağlantısı

Bu basit not defteri uygulaması, birçok web uygulamasında karşılaşılan dinamik davranışların temelini oluşturur. Örneğin:

- Alışveriş sepetine ürün eklemek/çıkarmak
- Görev listesi (to-do list) uygulamaları
- Mesajlaşma sistemlerinde gönderi ekleme ve silme

BASİT UYGULAMA ÖRNEKLERİ

Önceki bölümlerde DOM manipülasyonu ve olay yönetimi ayrıntılı olarak ele alınmış, web sayfalarının dinamik hale getirilmesinde kullanılan temel teknikler örneklerle açıklanmıştır. Ancak bu tekniklerin gerçek gücü, pratik uygulamalarda ortaya çıkar. Öğrenme sürecinde yalnızca teorik bilgiyi kavramak yeterli değildir; aynı zamanda bu bilgileri gerçek yaşam senaryolarında kullanabilmeniz beklenir.

Bu bölümde, JavaScript'in DOM manipülasyonu ve event handling yeteneklerini bir arada kullanan küçük ölçekli uygulamalar incelenecektir. Bu uygulamalar, daha büyük ve karmaşık web projelerinin temelini oluşturan yapı taşlarıdır. Aynı zamanda öğrencilere, basit kod parçacıklarıyla dahi kullanıcı deneyimini zenginleştirebileceklerini göstermektedir. Uygulama örnekleri, öğrencilerin en sık karşılaşacakları web işlevlerini temel alacak şekilde seçilmiştir. Bu kapsamda:

- **Dinamik Liste (To-Do List) Uygulaması:** Kullanıcıların görev ekleyip silebildiği bir yapılacaklar listesi.
- **Form Doğrulama Örneği:** Kullanıcı girişlerinin kontrol edilmesi ve uygun olmayan girişlerde uyarı verilmesi.
- **Görsel Galeri / Slayt Gösterisi:** Kullanıcının tıklamalarına veya gezinmelerine bağlı olarak görsellerin değişmesi.

Bu örnekler hem teknik becerilerin geliştirilmesine hem de problem çözme yetkinliklerinin artırılmasına hizmet edecektir. Ayrıca ilerleyen bölümlerde

işlenecek kütüphane ve çerçevelere (frameworks) geçiş için güçlü bir temel oluşturacaktır.

Dinamik Liste (To-Do List) Uygulaması

Giriş

Web uygulamalarında en çok kullanılan işlevlerden biri, kullanıcıların bir listeye öge ekleyebilmesi ve gerekirse bu öğeleri listeden çıkarabilmesidir. Günlük yapılacak işler listesi (to-do list) bunun en bilinen örneğidir. Bu uygulama, DOM manipülasyonu ve olay yönetiminin birlikte nasıl kullanılabileceğini gösteren ideal bir senaryodur. Öğrenciler bu örnek sayesinde hem yeni öge eklemeyi hem de öğeleri kaldırmayı pratik ederek, etkileşimli web tasarımının temelini pekiştirirler.

Uygulamanın Senaryosu

- Kullanıcı bir metin kutusuna görev (not) yazacaktır.
- "Ekle" butonuna basıldığında bu görev listeye eklenecektir.
- Listeye eklenen her ögenin yanında bir "Sil" butonu bulunacaktır.
- "Sil" butonuna tıklandığında ilgili öge listeden kaldırılacaktır.

Kod Örneği

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <title>To-Do List Uygulaması</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    #gorevListesi { margin-top: 15px; }
    li {
      background: #f9f9f9;
      padding: 10px;
      margin: 5px 0;
      border: 1px solid #ccc;
      border-radius: 4px;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    .sil {
      background: red;
      color: white;
      border: none;
      padding: 5px 10px;
      border-radius: 3px;
      cursor: pointer;
    }
  </style>
```

```

</head>
<body>
  <h2>Dinamik To-Do List</h2>
  <input type="text" id="gorev" placeholder="Görev girin">
  <button id="ekle">Ekle</button>
  <ul id="gorevListesi"></ul>
  <script>
    const ekleBtn = document.getElementById("ekle");
    const gorevListesi = document.getElementById("gorevListesi");
    ekleBtn.addEventListener("click", function() {
      const metin = document.getElementById("gorev").value.trim();
      if (metin !== "") {
        const li = document.createElement("li");
        li.textContent = metin;
        const silBtn = document.createElement("button");
        silBtn.textContent = "Sil";
        silBtn.classList.add("sil");
        silBtn.addEventListener("click", function() {
          li.remove();
        });
        li.appendChild(silBtn);
        gorevListesi.appendChild(li);
        document.getElementById("gorev").value = "";
      }
    });
  </script>
</body>
</html>

```

Uygulamanın Açıklaması

1. **Yeni öge oluşturma:** `document.createElement("li")` ile yeni liste ögesi oluşturulur.
2. **İçerik ekleme:** Kullanıcının yazdığı görev `textContent` ile liste ögesine eklenir.
3. **Silme işlevi:** Her ögeye bir "Sil" butonu eklenir ve bu butona tıklandığında ilgili öge DOM'dan kaldırılır.
4. **Dinamik yapı:** Liste, sayfa yenilenmeden güncellenir ve kullanıcı anlık olarak değişiklikleri görür.

Kendi Projende Dene

Bu uygulamayı daha ileri taşımak için aşağıdaki geliştirmeleri deneyiniz:

1. Listeye eklenen görevlerin tamamlandığını gösterecek bir özellik ekleyiniz. (İpucu: Görev tıklandığında yazının üstünü çizdirin.)
2. Görevleri tarayıcıda saklayınız. (İpucu: LocalStorage kullanarak sayfa yenilendiğinde görevlerin kaybolmamasını sağlayın.)
3. Her görevin yanına eklenme tarihini ve saatini gösteriniz. (İpucu: `Date()` nesnesinden yararlanabilirsiniz.)

Bu ödev sayesinde, JavaScript'in DOM manipülasyonu ve olay yönetimi becerilerini daha ileri düzeyde pekiştirmiş olacaksınız.

Form Doğrulama Uygulaması

Giriş

Web sitelerinde kullanıcıdan bilgi toplamak için en yaygın kullanılan yapı formlardır. Formlar sayesinde kullanıcılar üye olabilir, geri bildirim verebilir, ürün satın alabilir veya çeşitli verileri sisteme iletebilir. Ancak form verilerinin doğru ve güvenilir şekilde alınabilmesi için form doğrulama mekanizmaları gereklidir. Doğrulama, kullanıcıdan girilen verilerin belirli kurallara uygun olup olmadığını kontrol eder ve hatalı girişlerde kullanıcıya uyarı verir.

JavaScript ile yapılan form doğrulama işlemleri, kullanıcı deneyimini geliştiren ve sistemin güvenliğini artıran kritik bir adımdır. Örneğin, boş bırakılan alanların doldurulması, e-posta adresinin doğru formatta girilmesi veya şifrenin belirli kriterlere uygun olması gibi kontroller yapılabilir.

Örnek Uygulama: Basit Kayıt Formu

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <title>Form Doğrulama</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    .hata { color: red; font-size: 14px; }
    input { display: block; margin: 10px 0; padding: 5px; }
  </style>
</head>
<body>
  <h2>Kayıt Formu</h2>
  <form id="kayitFormu">
    <label>Ad:</label>
    <input type="text" id="ad" placeholder="Adınızı girin">
    <span id="adHata" class="hata"></span>
    <label>E-posta:</label>
    <input type="text" id="email" placeholder="E-posta adresiniz">
    <span id="emailHata" class="hata"></span>
    <label>Şifre:</label>
    <input type="password" id="sifre" placeholder="Şifreniz">
    <span id="sifreHata" class="hata"></span>
    <button type="submit">Gönder</button>
  </form>
  <script>
    const form = document.getElementById("kayitFormu");
    form.addEventListener("submit", function(event) {
      event.preventDefault(); // Sayfanın yenilenmesini engelle
```

```

    let valid = true;
    // Ad kontrolü
    const ad = document.getElementById("ad").value.trim();
    if(ad === "") {
        document.getElementById("adHata").textContent = "Ad alanı
boş bırakılamaz.";
        valid = false;
    } else {
        document.getElementById("adHata").textContent = "";
    }
    // E-posta kontrolü
    const email = document.getElementById("email").value.trim();
    const emailRegex = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
    if(!emailRegex.test(email)) {
        document.getElementById("emailHata").textContent = "Geçerli
bir e-posta giriniz.";
        valid = false;
    } else {
        document.getElementById("emailHata").textContent = "";
    }
    // Şifre kontrolü
    const sifre = document.getElementById("sifre").value;
    if(sifre.length < 6) {
        document.getElementById("sifreHata").textContent = "Şifre
en az 6 karakter olmalı.";
        valid = false;
    } else {
        document.getElementById("sifreHata").textContent = "";
    }
    if(valid) {
        alert("Form başarıyla gönderildi!");
    }
    });
</script>
</body>
</html>

```

Açıklama

- Kullanıcı formu doldurup "Gönder" butonuna bastığında JavaScript, alanları kontrol eder.
- Ad alanı boşsa kullanıcı uyarılır.
- E-posta adresi geçerli formatta değilse hata mesajı gösterilir.
- Şifre kısa ise en az altı karakterlik bir şifre girmesi istenir.
- Tüm kontroller geçildiğinde form başarıyla gönderilir.

Kendi Projende Dene

- Şifrenin en az bir büyük harf, bir küçük harf ve bir rakam içermesi kuralını ekleyiniz.

- E-posta adresi doğrulamasını daha kapsamlı hale getiriniz (örneğin .com, .edu.tr gibi uzantıları kontrol edin).
- Telefon numarası için ayrı bir alan ekleyip 10 haneli sayı olup olmadığını kontrol ediniz.

Bu geliştirmeler sayesinde form doğrulama becerilerinizi daha ileri seviyeye taşıyabilirsiniz.

Görsel Galeri / Slayt Gösterisi Uygulaması

Giriş

Modern web sitelerinin en dikkat çekici öğelerinden biri görsel galeriler ve slayt gösterileridir. Özellikle haber siteleri, e-ticaret platformları, portfolyo sayfaları veya eğitim sitelerinde görsellerin dinamik şekilde sunulması kullanıcı deneyimini zenginleştirir. Görsel galeri sayesinde kullanıcı, sayfadaki görselleri ileri-geri hareketlerle görüntüleyebilir, bazen de görseller otomatik olarak geçiş yapar.

JavaScript ile DOM manipülasyonu ve olay yönetimi kullanılarak basit ama işlevsel bir galeri veya slayt gösterisi oluşturmak mümkündür. Bu örnek, öğrencilerin hem içerik değiştirme hem de buton tıklama olaylarını yönetmeyi pratik etmelerine olanak tanır.

Örnek Uygulama: Basit Görsel Galeri

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <title>Görsel Galeri</title>
  <style>
    body { font-family: Arial, sans-serif; text-align: center;
margin: 20px; }
    #galeri {
      width: 400px;
      height: 250px;
      border: 2px solid #ccc;
      margin: auto;
      display: flex;
      justify-content: center;
      align-items: center;
    }
    img { max-width: 100%; max-height: 100%; }
    button {
      margin: 10px;
      padding: 8px 15px;
      border: none;
      background: #007BFF;
      color: white;
      border-radius: 5px;
    }
  </style>
</head>
<body>
  <div id="galeri">
    <img alt="Placeholder image for the gallery" data-bbox="280 500 680 800" />
  </div>
  <div>
    <button>Geri</button>
    <button>İleri</button>
  </div>
</body>
```

```

        cursor: pointer;
    }
    button:hover { background: #0056b3; }
</style>
</head>
<body>
    <h2>Basit Görsel Galerisi</h2>
    <div id="galeri">
        
    </div>
    <br>
    <button id="geri">Geri</button>
    <button id="ileri">İleri</button>
    <script>
        const gorsel = document.getElementById("gorsel");
        const geriBtn = document.getElementById("geri");
        const ileriBtn = document.getElementById("ileri");
        let index = 1;
        ileriBtn.addEventListener("click", function() {
            index++;
            if(index > 5) index = 1; // 5 görselden sonra başa dön
            gorsel.src = `https://picsum.photos/400/250?random=${index}`;
        });
        geriBtn.addEventListener("click", function() {
            index--;
            if(index < 1) index = 5; // geri gittiğinde sona dön
            gorsel.src = `https://picsum.photos/400/250?random=${index}`;
        });
    </script>
</body>
</html>

```

Açıklama

- **Başlangıç görseli:** Sayfa yüklendiğinde rastgele bir görsel gösterilir.
- **İleri butonu:** Kullanıcı "İleri" butonuna tıkladığında index değeri artar ve sonraki görsel yüklenir.
- **Geri butonu:** Kullanıcı "Geri" butonuna tıkladığında index değeri azalır ve önceki görsel yüklenir.
- **Sonsuz döngü:** Index değeri 5'i aştığında tekrar 1'e döner, 1'den küçüldüğünde ise 5'e gider.

Kullanım Alanları

- **E-ticaret siteleri:** Ürün fotoğraflarının galeri halinde sunulması.
- **Portfolyo sayfaları:** Sanatçı veya tasarımcının eserlerini sergilemesi.
- **Eğitim materyalleri:** Slayt tarzı görsellerle konu anlatımı.

Kendi Projende Dene

- Galeriye otomatik ilerleyen bir slayt gösterisine dönüştürünüz (ör. her 3 saniyede bir görsel değişsin).
- Küçük önizleme (thumbnail) görselleri ekleyerek kullanıcıya istediği görseli seçme imkânı sağlayınız.
- Her görsele açıklama metni (caption) ekleyiniz.

Bu eklemeler sayesinde, temel bir galeriyi daha gelişmiş bir kullanıcı deneyimine dönüştürebilirsiniz.

SONUÇ VE DEĞERLENDİRME

Bu ünite, JavaScript'in web sayfalarını statik bir yapıdan çıkararak etkileşimli ve dinamik bir yapıya dönüştürmedeki rolü açıklanmıştır. Öğrenciler, DOM manipülasyonu ile sayfa öğelerini seçme, değiştirme, ekleme ve silme işlemlerini öğrenmiş; event handling sayesinde kullanıcı eylemlerine nasıl tepki verileceğini kavramışlardır. Ayrıca basit uygulama örnekleri aracılığıyla bu bilgileri pratiğe dönüştürme fırsatı bulmuşlardır. Bu beceriler, modern web tasarımında kullanıcı deneyimini geliştirmek için kritik öneme sahiptir ve ilerleyen bölümlerde işlenecek kütüphane ve çerçeveler için gerekli altyapıyı sağlamaktadır.



Özet

• Bu ünite, web tasarımında etkileşimli yapılar oluşturma temel bileşenlerinden biri olan JavaScript'in ileri düzey kullanım alanları ele alınmıştır. Daha önceki ünitelerde HTML ve CSS ile statik içeriklerin oluşturulması, temel JavaScript yapılarıyla da basit düzeyde etkileşim sağlanması öğrenilmiştir. Bu ünite, ise öğrenciler, Document Object Model (DOM) manipülasyonu ve olay yönetimi (event handling) konularıyla tanıştırılmış; ayrıca bu tekniklerin nasıl gerçek uygulamalara dönüştürülebileceğini deneyimlemişlerdir. DOM, tarayıcı tarafından oluşturulan ve bir web sayfasındaki tüm öğeleri ağaç yapısı şeklinde temsil eden modeldir. Bu model sayesinde JavaScript, öğelere erişim sağlayabilir, içerikleri değiştirebilir, yeni öğeler ekleyebilir veya mevcut öğeleri kaldırabilir. DOM manipülasyonu, statik bir HTML sayfasını dinamik hale getirerek kullanıcının etkileşimlerine cevap verebilen bir yapı kazandırır. Bu bölümde getElementById, querySelector, getElementsByClassName, appendChild, remove gibi sık kullanılan yöntemler detaylı biçimde ele alınmıştır. Ayrıca innerHTML ile innerText arasındaki fark, classList ile stil değiştirme ve setAttribute ile öznelik güncelleme gibi işlemler örneklerle gösterilmiştir. Bir diğer önemli konu event handling, yani olay yönetimidir. Web sayfalarında kullanıcı eylemleri (fare tıklaması, klavye tuşu, form doldurma vb.) veya tarayıcı tarafından tetiklenen durumlar (sayfanın yüklenmesi, kaydırma, pencere boyutunun değişmesi) olay olarak tanımlanır. JavaScript ile bu olayların yakalanması ve uygun fonksiyonlarla işlenmesi, sayfaların etkileşimli hale gelmesini sağlar. Ünite, addEventListener metodunun modern yaklaşımı, olayların bubbling ve capturing aşamalarında nasıl işlendiği ve stopPropagation ile yayılımın nasıl kontrol edildiği açıklanmıştır. Özellikle click, hover, input, keydown gibi sık kullanılan olay türleri ayrıntılı olarak ele alınmış, küçük örneklerle pekiştirilmiştir. Ünite kapsamında, öğrencilerin teorik bilgileri pratiğe dönüştürmeleri için çeşitli uygulama örnekleri geliştirilmiştir. Bunlar arasında yapılacaklar listesi (to-do list), form doğrulama ve görsel galeri/slayt gösterisi yer almıştır. To-do list uygulaması, kullanıcıların listeye görev ekleyip silebilmesini sağlamış; form doğrulama uygulaması kullanıcıdan gelen bilgilerin kontrol edilmesinin önemini göstermiştir. Görsel galeri örneği ise ileri-geri butonlarıyla görsellerin değiştirilmesini sağlayarak DOM manipülasyonu ve event handling'in birlikte nasıl çalıştığını somutlaştırmıştır. Sonuç olarak bu ünite, öğrencilerin JavaScript bilgisini ileri düzeye taşıyarak, web tasarımında dinamik ve kullanıcı dostu sayfalar geliştirme becerisi kazanmalarını sağlamıştır. Öğrenciler, DOM manipülasyonu ve olay yönetiminin modern web uygulamalarında ne kadar merkezi bir role sahip olduğunu kavramış; küçük ölçekli projelerle bu bilgileri pekiştirmiştir. Böylece, ilerleyen bölümlerde ele alınacak olan kütüphane ve çerçevelere (jQuery, React, Bootstrap, Tailwind) geçiş için gerekli altyapı oluşturulmuştur.

DEĞERLENDİRME SORULARI

1. DOM'un açılımı aşağıdakilerden hangisidir?
 - a) Document Object Method
 - b) Data Object Model
 - c) Document Object Model
 - d) Dynamic Operation Module
 - e) Display Object Manager
2. getElementById("id") yöntemi ne döndürür?
 - a) Bir HTMLCollection
 - b) Tek bir öge
 - c) Bir NodeList
 - d) Tüm ögeler
 - e) Bir string
3. Aşağıdaki özelliklerden hangisi bir ögenin sadece metin içeriğini değiştirir, HTML etiketlerini işlemez?
 - a) innerText
 - b) innerHTML
 - c) style
 - d) classList
 - e) textCSS
4. document.querySelectorAll(".kutu") ifadesi ne döndürür?
 - a) Sadece ilk ögeyi
 - b) Bir string
 - c) Bir NodeList
 - d) Bir HTMLCollection
 - e) Bir boolean

5. Bir ögenin sınıf listesinden mevcut bir sınıfı kaldırmak için hangi yöntem kullanılır?
 - a) `classList.add()`
 - b) `classList.toggle()`
 - c) `className = ""`
 - d) `classList.remove()`
 - e) `classListDrop()`

6. Form alanı odaklandığında tetiklenen olay aşağıdakilerden hangisidir?
 - a) `blur`
 - b) `input`
 - c) `change`
 - d) `load`
 - e) `focus`

7. `event.stopPropagation()` ne işe yarar?
 - a) Olayı yeniden başlatır
 - b) Olayın bubbling aşamasını durdurur
 - c) Olayı tekrar tetikler
 - d) Olayı tarayıcıdan siler
 - e) Olayı capturing aşamasına taşır

8. Aşağıdaki olaylardan hangisi bir form gönderildiğinde tetiklenir?
 - a) `submit`
 - b) `load`
 - c) `change`
 - d) `input`
 - e) `focus`

9. Aşağıdaki kod parçası çalıştırıldığında çıktısı ne olur?

```
let div = document.createElement("div");  
div.innerHTML = "<p>Paragraf</p>";  
console.log(div.textContent);
```

- a) <p>Paragraf</p>
- b) div
- c) null
- d) Hata verir
- e) Paragraf

10. Bir öğeye aynı anda hem click hem de dblclick olayları tanımlandığında hangi senaryo gerçekleşir?

- a) Sadece click çalışır
- b) Sadece dblclick çalışır
- c) Olaylar çalışır ve hata verir
- d) Önce click, ardından dblclick çalışır
- e) Hiçbiri çalışmaz

Cevap anahtarı

1. c, 2. b, 3. a, 4. c, 5. d, 6. e, 7. b, 8. a, 9. e, 10. d

YARARLANILAN KAYNAKLAR

Ducket, J., Moore J., Ruppert, G. & Stone, E. (2014). *HTML & CSS design and build websites. JavaScript & JQuery interactive front-end web development First edition*, Wiley.

Flanagan, D. (2020). *JavaScript: The Definitive Guide: Master the World's Most-Used Programming Language 7th Edition*, O'Reilly Media, Inc.

Frisbie, M. (2023). *Professional JavaScript for Web Developers Fifth Edition*, Wiley.

Resig, J., Bibeault, B. & Maras, J. (2016), *Secrets of the javascript Ninja 2nd Edition*, Manning Publications.

Sambells, J. & Keith, J. (2010) *DOM Scripting: Web Design with JavaScript and the Document Object Model, Second Edition*, Apress.